

Documentação completa do projeto OpenMovie

Especificação do MVP, integração OMDb, testes e entrega

Autores

Felipe Gustavo Bordinon

Matheus Augusto Filippi Polinski

Data: 30/09/2026

Versão: 3.0

Status: MVP em desenvolvimento

Documento sanitizado

Nenhuma chave real aparece nesta versão. Exemplos usam SUA_CHAVE; a credencial é fornecida localmente por `--dart-define`.

Aplicação Flutter para consulta de filmes e séries. A prioridade atual é validar o funcionamento antes de desenvolver o design.

1. Visão, objetivo e escopo

O OpenMovie consulta metadados de obras audiovisuais na OMDb por HTTPS. O app começa com uma consulta fixa e evolui para busca, detalhes, filtros e séries.

1.1 Objetivos

- Comprovar a comunicação HTTP entre Flutter e OMDb.
- Exibir respostas e erros sem interromper o aplicativo.
- Manter o código simples e compatível com o conteúdo estudado em aula.
- Documentar execução, dependências, segurança, testes e entrega.

1.2 Escopo por etapa

Etapa	Funcionalidades	Situação
1 - MVP	t=Batman, JSON, Map, Response, Title e erros.	Implementado
2 - Pesquisa	Campo de busca, s e lista Search.	Planejado
3 - Detalhes	Consulta por imdbID com i e plot=full.	Planejado
4 - Refinamento	type, y, page e totalResults.	Planejado
5 - Séries	Season, Episode e lista Episodes.	Planejado
6 - Qualidade	Mais testes, acessibilidade e design.	Planejado

1.3 Limites atuais

- Sem backend próprio, login, streaming, comentários ou recomendações.
- Poster API separada não será usada; o campo Poster da resposta principal é suficiente.
- Plataforma final, favoritos e identidade visual permanecem a definir.
- Funcionalidade planejada só será marcada como concluída após execução e teste.

História principal

Como pessoa interessada em filmes e séries, quero pesquisar uma obra e consultar seus dados para encontrar rapidamente informações confiáveis.

2. Arquitetura, ambiente e segurança

2.1 Fluxo do MVP

```
Tela -> Uri.https -> http.get -> JSON -> Map<String, dynamic>
      -> Response=True -> Title -> setState -> Tela
```

O MVP pode permanecer em lib/main.dart. Quando as novas telas forem implementadas, a separação sugerida é: telas/, serviços/ e modelos/.

2.2 Ambiente verificado

Componente	Configuração
Flutter	3.44.2 stable
Dart	3.12.2; projeto sdk: ^3.12.0
HTTP	http: ^1.6.0
Qualidade	flutter_test e flutter_lints: ^6.0.0
Android	android.permission.INTERNET presente

2.3 Execução

```
flutter pub get
flutter run --dart-define=OMDB_API_KEY=SUA_CHAVE

dart format lib test
flutter analyze
flutter test
```

2.4 Regras da chave

- Ativar a chave recebida por e-mail e passá-la apenas localmente.
- Nunca registrar a URL completa quando ela contiver apikey.
- Não colocar chave ou link de ativação em código, README, PDF, print ou commit.
- Se houver exposição pública, substituir a chave antes da entrega.

```
const chaveApiOmdb = String.fromEnvironment('OMDB_API_KEY');
```

Validação atual

flutter analyze terminou sem problemas e flutter test foi aprovado. A validação visual com chave ativa ainda deve preceder o print.

3. Contrato da OMDb API

Propriedade	Definição
Base	https://www.omdbapi.com/
Método	GET por HTTPS
Autenticação	apikey obrigatório
Formato	JSON por padrão; XML com r=xml
Limite gratuito	1.000 consultas por dia, conforme cadastro oficial
Sucesso	HTTP 200 e Response igual a True
Erro	Response igual a False e campo Error

3.1 Parâmetros

Parâmetro	Valores	Uso
apikey	Chave ativa	Obrigatório em todas as consultas.
t	Texto	Título para resposta detalhada.
i	IMDb ID	Identificador para resposta detalhada.
s	Texto	Pesquisa que retorna uma lista.
type	movie, series, episode	Filtro de tipo.
y	Ano	Filtro de lançamento.
plot	short, full	Tamanho da sinopse; padrão short.
r	json, xml	Formato; padrão json.
page	1 a 100	Página da pesquisa; padrão 1.
Season	Número	Temporada de uma série.
Episode	Número	Episódio da temporada.
callback / v	JSONP / 1	Não usados no app; v é reservado.

3.2 Regras

- Consulta detalhada exige t ou i; pesquisa em lista exige s.
- Preferir Uri.https para escapar títulos e parâmetros.
- Verificar status HTTP, depois JSON, Response e Error.
- Campos podem estar ausentes ou conter N/A.

4. Implementação do MVP atual

4.1 Requisição sanitizada

```
https://www.omdbapi.com/?apikey=SUA_CHAVE&t=Batman&r=json
```

4.2 Respostas mínimas

SUCESSO	ERRO
<pre>{ "Title": "Batman", "Response": "True" }</pre>	<pre>{ "Response": "False", "Error": "Movie not found!" }</pre>

4.3 Leitura usada no projeto

```
final resposta = await http.get(url);  
if (resposta.statusCode != 200) throw Exception('Erro HTTP');  
  
final Map<String, dynamic> dados = json.decode(resposta.body);  
final bool sucesso = dados['Response'] == 'True';  
  
if (sucesso) {  
  tituloFilme = dados['Title'] ?? 'Título indisponível';  
} else {  
  mensagem = dados['Error'] ?? 'Erro na consulta';  
}
```

4.4 Comportamento obrigatório

- 1 Ao iniciar, conferir se OMDB_API_KEY foi informada.
- 2 Montar a URI com apikey, t=Batman e r=json.
- 3 Enviar GET e exigir status HTTP 200.
- 4 Converter o corpo para Map.
- 5 Exibir Title em sucesso ou uma mensagem legível em falha.
- 6 Antes de setState após await, conferir mounted.

Tela atual

AppBar Open Movie Database, identificação da consulta por Batman, título retornado e mensagem de sucesso ou erro.

5. Pesquisa, paginação e detalhes

5.1 Pesquisa planejada

```
?apikey=SUA_CHAVE&s=Batman&type=movie&page=1&r=json
```

Campo	Tipo	Uso
Search	List<dynamic>	Lista resumida.
Search[].Title	String	Título no item.
Search[].Year	String	Ano ou intervalo.
Search[].imdbID	String	Abrir detalhes.
Search[].Type	String	movie, series ou episode.
Search[].Poster	String	URL ou N/A.
totalResults	String	Total de correspondências.
Response / Error	String	Sucesso e motivo da falha.

Regras de pesquisa

- Não enviar texto vazio; remover espaços; nova busca começa na página 1.
- Durante carregamento, impedir envios duplicados; Search ausente não vira lista.
- Pagarinar somente entre 1 e 100 e não misturar pesquisas diferentes.

5.2 Detalhes por identificador

```
?apikey=SUA_CHAVE&i=IMDB_ID&plot=full&r=json
```

Grupo	Campos previstos
Identificação	Title, Year, Rated, Released, Runtime, Genre, Type, imdbID
Conteúdo	Director, Writer, Actors, Plot, Language, Country, Awards, Poster
Avaliações	Ratings[], Metascore, imdbRating, imdbVotes
Adicionais	DVD, BoxOffice, Production, Website, Response

- A presença dos campos varia; validar ausência e N/A antes de exibir.
- Ratings e valores numéricos chegam como texto e podem usar escalas diferentes.
- Usar ícone substituto quando Poster for N/A ou falhar.

6. Séries, episódios e tratamento dos dados

6.1 Consultas planejadas

TEMPORADA: ?apikey=SUA_CHAVE&i=ID_DA_SERIE&Season=1&r=json
EPISÓDIO: ?apikey=SUA_CHAVE&i=ID_DA_SERIE&Season=1&Episode=1&r=json

Campo	Tipo	Uso
totalSeasons	String	Total de temporadas.
Season	String	Temporada consultada.
Episodes	List<dynamic>	Lista de episódios.
Episodes[].Title	String	Título.
Episodes[].Released	String	Data.
Episodes[].Episode	String	Número.
Episodes[].imdbRating	String	Nota ou N/A.
Episodes[].imdbID	String	Abrir ficha completa.
seriesID	String	Série vinculada ao episódio.

6.2 Estruturas Dart

Estrutura	Uso
Map<String, dynamic>	Objeto JSON detalhado e acesso por nome.
List<dynamic>	Search, Ratings e Episodes.
String	Campos, mensagens, números e datas retornados como texto.
bool	Estados da tela e resultado de Response == True.

6.3 Regras de dados

- Usar nomes exatamente como retornados: Title, imdbID, imdbRating e Response.
- Não converter notas, votos ou totais antes de validar o conteúdo.
- Ocultar campo opcional vazio ou mostrar Informação não disponível.
- Não depender de todos os textos permanecerem iguais ao longo do tempo.

7. Telas, estados, erros e segurança

Tela	Dados principais	Ações
MVP	Title, Response, Error	Carregar Batman automaticamente.
Busca	Search[], totalResults	Pesquisar, paginar e abrir resultado.
Detalhes	Ficha técnica, Plot, Poster, Ratings	Voltar e exibir campos disponíveis.
Série	totalSeasons e Episodes[]	Escolher temporada e episódio.

7.1 Estados obrigatórios

Estado	Comportamento
Inicial	Orientação ou consulta fixa.
Carregando	Progresso e bloqueio de repetição.
Sucesso	Conteúdo com campos disponíveis.
Vazio	Mensagem sem lista inválida.
Erro	Mensagem amigável e nova tentativa.
Imagem indisponível	Ícone ou espaço reservado.

7.2 Matriz de falhas

Falha	Deteção	Resposta
Sem chave	OMDB_API_KEY vazia	Orientar execução; não consultar.
Chave inválida/limite	Response=False e Error	Informar sem expor credencial.
HTTP	statusCode diferente de 200	Erro de comunicação.
Rede/JSON	Exceção	Mensagem e app responsivo.
Campo ausente/N/A	Validação por campo	Ocultar ou substituir.
Widget fechado	mounted=false	Não executar setState.

7.3 Segurança e privacidade

- Usar HTTPS; enviar à OMDb apenas chave e parâmetros necessários.
- Não coletar conta, e-mail ou localização no MVP.
- Revisar git diff e prints para impedir vazamento de chave ou informação pessoal.

8. Testes, aceite e entrega

8.1 Cobertura

Nível	Cobertura
Widget atual	AppBar e identificação da consulta fixa.
Unidade planejada	Sucesso, Error, campos ausentes e N/A.
Widget planejado	Carregando, vazio, erro e navegação.
Serviço planejado	URI para t, s, i, filtros e páginas.
Manual	Chave válida/inválida, sem rede, resultado e pôster.

8.2 Critérios de aceite do MVP

- 1 flutter pub get resolve as dependências.
- 2 flutter analyze termina sem problemas e flutter test é aprovado.
- 3 Com chave ativa e rede, a tela mostra o Title retornado para Batman.
- 4 Sem chave ou em falha, a tela mostra mensagem e não encerra o app.
- 5 Nenhuma chave real aparece nos arquivos destinados ao GitHub.

8.3 Entrega parcial

Item	Evidência	Situação
Código documentado	README com objetivo, execução, API e estrutura.	Preparado localmente
Lista de pendências	Checklist no README e neste documento.	Preparada
Print de todas as telas	PNG legível de cada tela implementada.	Pendente

8.4 Checklist antes do commit

- Formatar, analisar, testar e executar o aplicativo com chave ativa.
- Confirmar que código, documentação e prints não contêm credenciais.
- Adicionar somente arquivos da atividade e manter recursos futuros como pendentes.
- Salvar prints em docs/screenshots, com nomes como 01_mvp_batman.png.

Requisito de print

Para o MVP atual, o print mínimo deve mostrar a tela com o título retornado. Novas telas exigirão novos prints.

9. Pendências, decisões e referências

9.1 Backlog priorizado

Prioridade	Item	Dependência
Alta	Executar MVP e gerar print.	Chave ativa e dispositivo.
Alta	Publicar PDF sanitizado e README.	Revisão sem credenciais.
Média	Busca por s e lista Search.	Aprovação da etapa.
Média	Detalhes por i e plot=full.	imdbID disponível.
Média	Filtros type e y.	Tela de busca definida.
Baixa	Paginação e séries.	Fluxos anteriores validados.
Posterior	Design e acessibilidade.	MVP aprovado.

9.2 Decisões abertas

- Plataforma principal: Android, web, Windows ou múltiplas.
- Busca: somente filmes ou também séries e episódios.
- Filtros e paginação obrigatórios; inclusão ou não de favoritos.
- Tratamento visual para obras sem pôster e início da etapa de design.

9.3 Referências oficiais

Fonte	Endereço
OMDb - uso e parâmetros	https://www.omdbapi.com/
OMDb - cadastro	https://www.omdbapi.com/apikey.aspx
OMDb - Swagger	https://www.omdbapi.com/swagger.json
OMDb - repositório	https://github.com/OMDbAPI/OMDb-API
Dart - JSON	https://api.dart.dev/dart-convert/jsonDecode.html
Flutter - requisições	https://docs.flutter.dev/cookbook/networking/fetch-data
Pacote http	https://pub.dev/packages/http

9.4 Rastreabilidade do MVP

Requisito	Implementação
Consulta fixa	t=Batman e r=json em buscarFilme().
Chave	OMDB_API_KEY por dart-define.
Resposta	HTTP, Map, Response, Title e Error.
Internet	Permissão no AndroidManifest.xml.
Qualidade	flutter analyze e flutter test.
Entrega	README, pendências, PDF e print.
Estado	A documentação cobre o MVP e as evoluções previstas. Itens ainda não definidos permanecem explícitos para não transformar hipóteses em requisitos.